

第8章 二分图、平面图和树

本章将讨论几类在理论研究和实际应用中都有重要意义的无向图，它们是二分图、平面图和树。

8.1 二分图

8.1.1 二分图的基本概念

定义 8.1 无向图 $G = \langle V, E \rangle$ 称为二分图 (bipartite graph)，如果有非空集合 X, Y 使 $X \cup Y = V, X \cap Y = \emptyset$ ，且对每一 $e \in E$ ，都有 $e = \{x, y\}, x \in X, y \in Y$ 。此时常用 $\langle X, E, Y \rangle$ 表示二分图 G 。若对 X 中任一 x 及 Y 中任一 y 恰有一边 $e \in E$ ，使 $e = \{x, y\}$ ，则称 G 为完全二分图 (complete bipartite graph)。当 $|X| = m, |Y| = n$ 时，完全二分图 G 记为 $K_{m,n}$ 。

例 8.1 图 8.1 中 (a), (b) 为二分图, (c) 为完全二分图 $K_{3,3}$, (d), (e) 不是二分图。

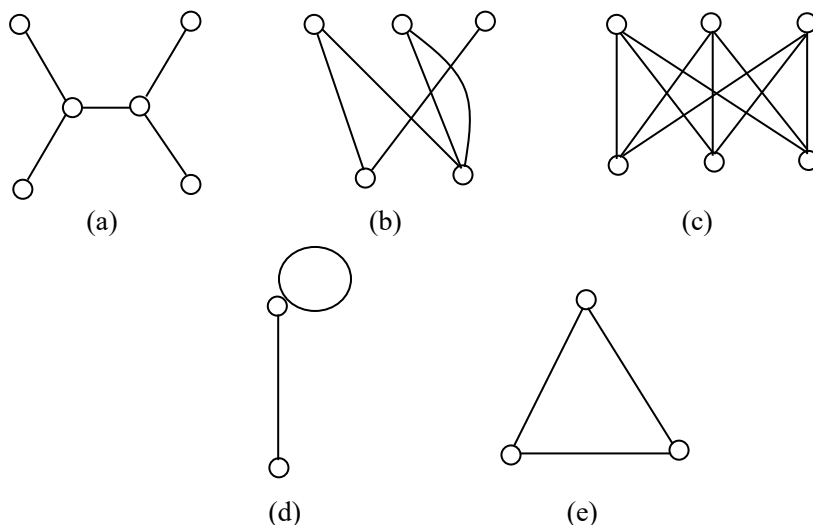


图 8.1

二分图的下列特征性是重要的。

定理 8.1 无向图 G 为二分图的充分必要条件是, G 至少有两个顶点, 且其所有回路的长度均为偶数。

证 先证必要性。

设 G 为二分图 $\langle X, E, Y \rangle$ 。由于 X, Y 非空, 故 G 至少有两个顶点。若 C 为 G 中任一长度为 l 回路, 令

$$C = (v_0, v_1, v_2, \dots, v_{l-1}, v_l = v_0)$$

其中诸 $v_i (i = 0, 1, \dots, l)$ 必定相间出现于 X 及 Y 中, 显然

$$\{v_0, v_2, v_4, \dots, v_l = v_0\} \subseteq X$$

$$\{v_1, v_3, v_5, \dots, v_{l-1}\} \subseteq Y$$

因此 l 必为偶数, 从而 C 中有偶数条边。

再证充分性。

设 G 的所有回路具有偶数长度, 并设 G 为连通图 (不失一般性, 若 G 不连通, 则可对 G 的各连通分支作下述讨论)。

令 G 的顶点集为 V , 边集为 E , 现构造 X, Y , 使 $\langle X, E, Y \rangle = G$ 。取 $v_0 \in V$, 置

$$X = \{v \mid v = v_0 \text{ 或 } v \text{ 到 } v_0 \text{ 有偶数长度的通路}\}$$

$$Y = V - X$$

X 显然非空。现需证 Y 非空, 且没有任何边的两个端点都在 X 中或都在 Y 中。

若 v_0 无相邻顶点, 那么 X 中无其他顶点, 由于 $|V| \geq 2$, 故至少还有一个顶点 v 在 Y 中;

若 v_0 有相邻顶点 v_1 , 那么 $v_1 \in Y$; 故 Y 不空。

设有边 $\{u, v\}$, 使 $u \in X, v \in X$ 。那么, v_0 到 u 有偶数长度的通路, 或 $u = v_0$; v_0 到 v 有偶数长度的通路, 或 $v = v_0$ 。这样, 无论何种情况, 连同边 $\{u, v\}$ 均有一条从 v_0 到 v_0 的奇数长度的闭的拟路径, 在此闭的拟路径中一定存在奇数长度的回路 (回忆定理 8.5 否则该闭路径的长度为偶数), 与题设矛盾。故不可能有边 $\{u, v\}$ 使 u, v 均在 X 中。

仿上可证 “没有任何边的两个端点全在 Y 中”。

作为一种数学模型二分图是十分有用的, 许多问题可以用它来刻画。例如 “资源分配”、“工作安排”、“人员择偶” 等等。但是, 利用二分图分析解决这类问题时, 还需要有关二分图的另一个概念——匹配。

8.1.2 匹配

定义 8.2 设 $G = \langle X, E, Y \rangle$ 为二分图, $M \subseteq E$ 。称 M 为 G 的一个**匹配** (matching), 如果 M 中任何两条边都没有公共端点。 $M = \emptyset$ 时称 M 为空匹配。 G 的所有匹配中边数最多的匹配称为**最大匹配** (maximal matching)。如果 $X(Y)$ 中任一顶点均为匹配 M 中边的端点, 那么称 M 为 $X(Y)$ -**完全匹配** (perfect matching)。若 M 既是 X -完全匹配又是 Y -完全匹配, 则称 M 为 G 的**完全匹配**。

例 8.2 图 8.2 中各图的粗线表示匹配中的边 (简称匹配边)。(b) 中匹配是最大的, $X = \{a, b, c\}$, 匹配是 X -完全的, (c) 中匹配是 G 的完全匹配 (从而也是最大的)。

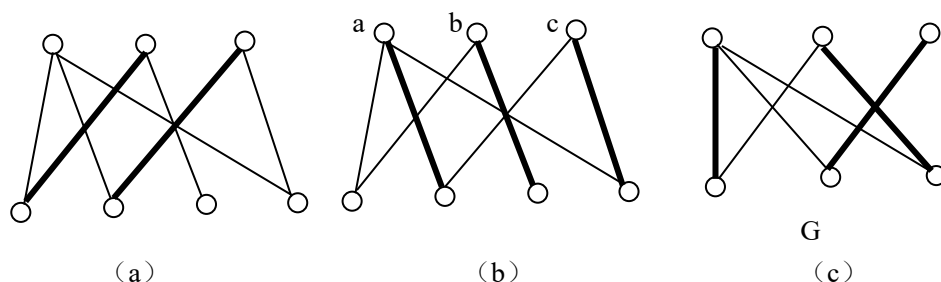


图 8.2

注意: 最大匹配总是存在但未必唯一; $X(Y)$ -完全匹配及 G 的完全匹配必定是最大的, 但反之则不然; $X(Y)$ -完全匹配未必存在。

为讨论最大匹配的求法及完全匹配的存在条件, 需要下列术语。

定义 8.3 设 $G = \langle X, E, Y \rangle$, M 为 G 的一个匹配。

(1) M 中边的端点称为 **M-顶点**, 其它顶点称为**非 M-顶点**。

(2) G 中 v_k 到 v_l 的通路 P 称为**交替链**, 如果 P 的起点 v_k 和终点 v_l 为非 M -顶点, 而其边的序列中非匹配边与匹配边交替出现 (从而首尾两边必为非匹配边, 除顶点 v_k, v_l 以外各顶点均为 M -顶点)。特别地, 当一边 $\{v, v'\}$ 两端点均为非 M -顶点, 通路 $\{v, v'\}$ 亦称为交替链。

以下算法可把 G 中任一匹配 M 扩充为最大匹配, 此算法被称为**匈牙利算法**:

(1) 首先用 (*) 标记 X 中所有的非 M -顶点, 然后交替进行步骤 (2), (3)。

(2) 选取一个刚标记 (用 (*) 或在步骤 (3) 中用 (y_i) 标记) 过的 X 中顶点, 例如顶点 x_i , 然后用 (x_i) 去标记 Y 中顶点 y , 如果 x_i 与 y 为同一非匹配边的两端点, 且在本步骤中 y 尚未被标记过。重复步骤 (2), 直至对刚标记过的 X 中顶点全部完成一遍上述过程。

(3) 选取一个刚标记 (在步骤 (2) 中用 (x_i) 标记) 过的 Y 中结点, 例如 y_i , 用 (y_i) 去标记 X 中结点 x , 如果 y_i 与 x 为同一匹配边的两端点, 且在本步骤中 x 尚未被标记过。重复步骤 (3), 直至对刚标记过的 Y 中结点全部完成一遍上述过程。

(2), (3) 交替执行, 直到下述情况之一出现为止:

(I) 标记到一个 Y 中顶点 y , 它不是 M -顶点。这时从 y 出发循标记回溯, 直到 (*)

标记的 X 中顶点 x ，我们求得一条交替链。设其长度为 $2k+1$ ，显然其中 k 条是匹配边， $k+1$ 条是非匹配边。

(II) 步骤 (2) 或 (3) 找不到可标记结点，而又不是情况 (I)。

(4) 当 (2), (3) 步骤中断于情况 (I)，则将交替链中非匹配边改为匹配边，原匹配边改为非匹配边 (从而得到一个比原匹配多一条边的新匹配)，回到步骤 (1)，同时消除一切现有标记。

(5) 对一切可能，(2) 和 (3) 步骤均中断于情况 (II)，或步骤 (1) 无可标记结点，算法终止 (算法找不到交替链)。

我们只用一个例子跟踪算法的执行，不打算证明算法的正确性。

例 8.3 用匈牙利算法求图 8.3 的一个最大匹配。

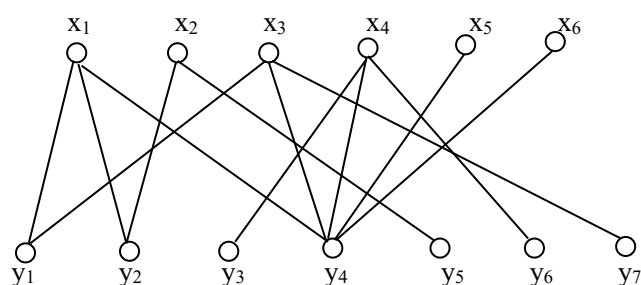


图 8.3

(1) 置 $M = \emptyset$ ，对 x_1 - x_6 标记 (*)。

(2) 找到交替链 (x_1, y_1) (由标记 (x_1) , (*) 回溯得)，置 $M = \{\{x_1, y_1\}\}$ 。

(3) 找到交替链 (x_2, y_2) (由标记 (x_2) , (*) 回溯得)，置 $M = \{\{x_1, y_1\}, \{x_2, y_2\}\}$ 。

(4) 找到交替链 (x_3, y_1, x_1, y_4) (如图 8.4 所示。图中虚线表示非匹配边，细实线表示交替链中非匹配边，粗实线表示匹配边)，因而得 $M = \{\{x_2, y_2\}, \{x_3, y_1\}, \{x_1, y_4\}\}$ 。

(5) 找到交替链 (x_4, y_3) (由标记 (x_4) , (*) 回溯得)，置 $M = \{\{x_2, y_2\}, \{x_3, y_1\}, \{x_1, y_4\}, \{x_4, y_3\}\}$ 。

(6) 找到交替链 $(x_5, y_4, x_1, y_1, x_3, y_7)$ (如图 8.5 所示，图中各种线段的意义同上)，因而得

$$M = \{\{x_2, y_2\}, \{x_4, y_3\}, \{x_5, y_4\}, \{x_1, y_1\}, \{x_3, y_7\}\}$$

即为最大匹配 (如图 8.6 所示)。

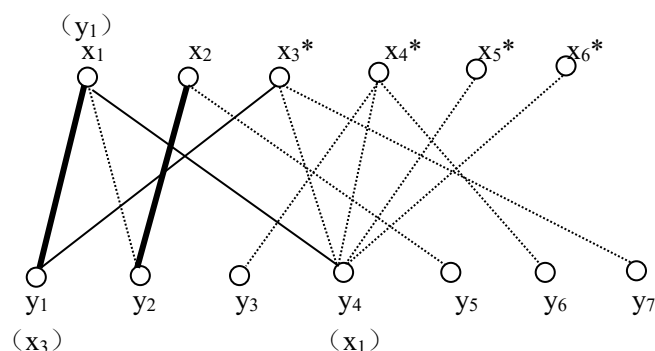


图 8.4

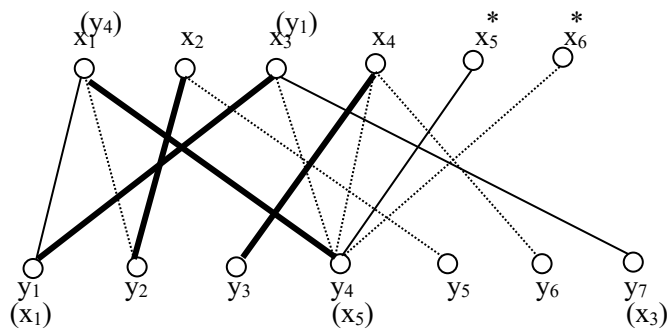


图 8.5

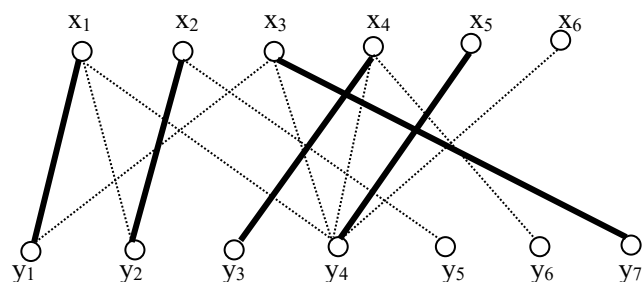


图 8.6

现在我们来讨论完全匹配的存在条件。为此，定义图 $G = \langle V, E, \Psi \rangle$ 的顶点子集 $S \subseteq V$ 的相邻顶点集 $N(S)$ （所有与 S 中顶点相邻的顶点组成的集合）：

$$N(S) = \{v \mid v \in V \wedge \exists u \exists e (u \in S \wedge e \in E \wedge e = \{u, v\})\}$$

或

$$N(S) = \{v \mid \exists u \exists e (u \in S \wedge e = \{u, v\})\}$$

定理 8.2 设图 $G = \langle X, E, Y \rangle$ 。G 有 X -完全匹配的充分必要条件是：对每一 $S \subseteq X$ 有

$$|N(S)| \geq |S|$$

此定理是霍尔（Hall）于 1935 年证明的，通常称为霍尔（婚姻）定理。

证 必要性是易证的。设 G 有 X -完全匹配

$$\{\{x_0, y_{i0}\}, \{x_1, y_{i1}\}, \dots, \{x_m, y_{im}\}\} \quad (m = |X|)$$

其中诸 x_i 互不相同，诸 y_{ij} 也各不一样，因此对任一 $S \subseteq X$ ， S 中有多少元素， $N(S)$ 中至少也有同样多的元素，即 $|N(S)| \geq |S|$ 。

现证充分性。

设 G 满足：对任一 $S \subseteq X$ ， $|N(S)| \geq |S|$ ，但 G 无 X -完全匹配。

作 G 的最大匹配 M ，据假设，至少有顶点 $x \in X$ ， x 不是 M -顶点。用匈牙利算法求起始于 x 的所有交替链。由于 M 已是最大匹配，上述构造过程必定在步骤（3）中因情况（II）停止，即它们都只能是末尾边为匹配边的链（暂称伪交替链）。用 Q 表示这些伪交替链上的顶点集合，易知， Q 中只有 x 不是 M -顶点。置

$$S = Q \cap X, S' = Q \cap Y \quad (\text{参阅图 8.7})$$

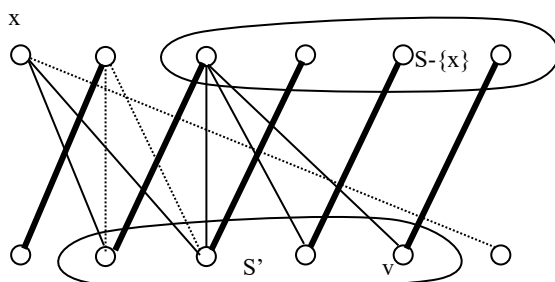


图 8.7

显然, $S - \{x\}$ 中顶点与 S' 中顶点一一对应, 因此

$$|S'| = |S| - 1$$

$$S' \subseteq N(S)$$

事实上 $N(S) = S'$, 因为对每一 $y \in N(S)$, 都有顶点 $u \in S = Q \cap X$ (因而 $u \in Q$) 使 $\{u, y\}$ 为一伪交替链中的边, 从而 $y \in Q$, 即 $y \in Q \cap Y = S'$ 之中。据以上讨论,

$$|N(S)| = |S'| = |S| - 1$$

$$|N(S)| < |S|$$

与题设矛盾, 因此 M 必是 X -完全匹配。

定理 8.2 证毕。对 Y -完全匹配有类似的结论, 不赘。

例 8.4 k -正则二分图 ($k > 0$) 有完全匹配。

证 设 $G = \langle X, E, Y \rangle$ 为 k -正则二分图, 那么

$$k \cdot |X| = |E| = k \cdot |Y|$$

从而 $|X| = |Y|$ 。

设 S 为 X 的任一子集, E_1 为 S 中顶点所关联的边的集合, E_2 为 $N(S)$ 中顶点所关联的边的集合。据 $N(S)$ 的定义, $E_1 \subseteq E_2$, 于是

$$k \cdot |N(S)| = |E_2| \geq |E_1| = k \cdot |S|$$

$$|N(S)| \geq |S|$$

因此 G 有 X -完全匹配 M 。

由于 $|X| = |Y|$, 显然 M 也是 Y -完全匹配, 故 G 有完全匹配。

8.2 平面图

8.2.1 平面图的基本概念

许多实际问题可以抽象为这样的模式：在一些表示客体的结点之间“布线”，以建立它们之间的某种联系，要求这些线在一个平面上而又不相互交叠。这正是一个本节要讨论的图论问题。

例如，要在三个工作点 A, B, C 和三个原料库 L, M, N 之间建立各工作点到各原料库的传输线，问是否可能使这些线路互不相交？如果用结点表示工作点，用边表示传输线，那么上述问题便可描述为：图 $K_{3,3}$ 是否可以在一个平面上图示出来，而使图中各边除端点外均不相交？另外印刷线路板上的布线、交通道路的设计等都是此类问题。为了深入讨论这类问题，需要引入平面图的概念。

定义 8.4 无向图 G 称为**平面图** (planar graph)，如果 G 可以在一个平面上图示出来，而使各边仅在顶点处相交。否则称 G 为**非平面图**。

例 8.5

(1) 图 8.10 中除(e)外各图均为平面图。(a)，(b) 显然为平面图；(c) 为平面图，因为它可以图示如 (d)。

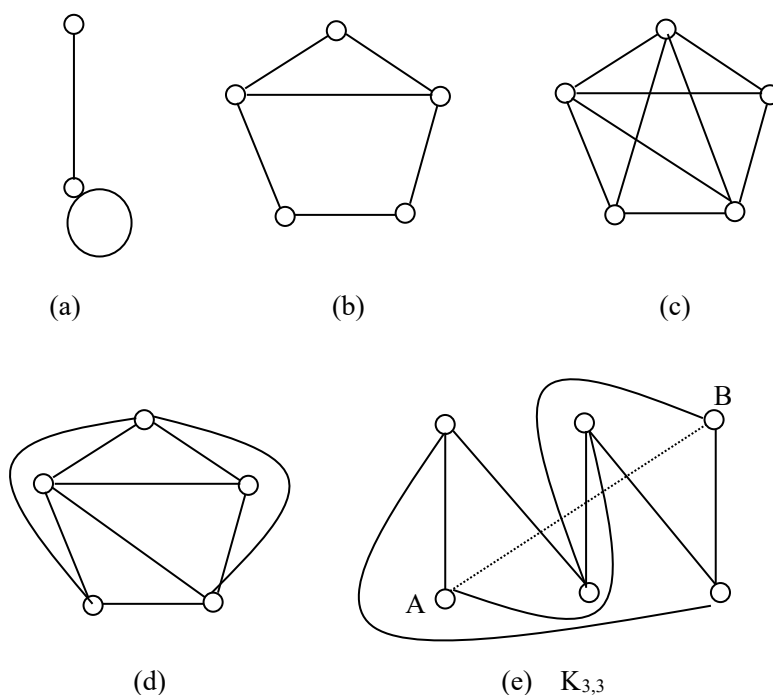


图 8.10

(2) $K_{3,3}$, K_5 都不是平面图。图 8.10 (e) 中边 (A, B) 无论如何将与其它边相交（指不在顶点处相交，下同），因此 $K_{3,3}$ 非平面图。图 8.10 (d) 中若再添边（非平行边和环）即为 K_5 ，这一新添加的边无论如何也将与其它边相交，因此 K_5 也不是平面图。当然这种解释不是严格证明！下文将给出严格证明。

$K_{3,3}$ 与 K_5 称为**库拉托夫斯基 (Kuratowski) 图**，它们有一些有趣的共同点：

(1) 它们都是正则图。

(2) 去掉一条边时它们都是平面图。

(3) $K_{3,3}$ 是边数最少的非平面简单图, K_5 是顶点数最少的非平面简单图, 因而它们都是最基本的非平面图。

平面图除了有顶点、边的概念, 还有面的概念, 为简单计, 只对平面连通图讨论之。

定义 8.5 平面连通图中各边所界定的区域称为平面图的面 (regions)。有界的区域称为有界面, 无界的区域称为无界面。界定各面的闭的拟路径称为面的边界 (boundary), 它的长度称为面的度 (degree)。

例 8.6 图 8.11 为一平面图, 它共有 5 个面 r_1, r_2, r_3, r_4, r_5 , 其中 r_5 为无界面, 其余为有界面, 它们的边界分别是

r_1 : 界定该面的闭的拟路径 (v_1, v_1) , 度为 1。

r_2 : 界定该面的闭的拟路径 (v_2, v_3, v_2) , 度为 2

r_3 : 界定该面的闭的拟路径 (v_4, v_5, v_8, v_4) , 度为 3

r_4 : 界定该面的闭的拟路径 $(v_5, v_6, v_7, v_6, v_8, v_5)$, 度为 5

r_5 : 界定该面的闭的拟路径 $(v_1, v_1, v_2, v_3, v_4, v_5, v_6, v_8, v_4, v_3, v_2, v_1)$, 度为 11。

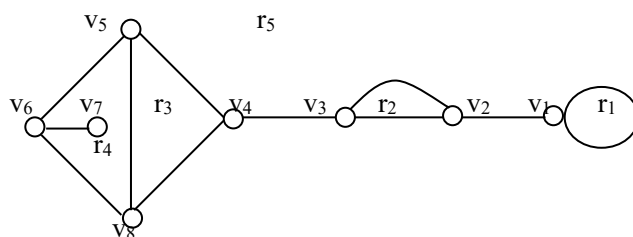


图 8.11

下面常讨论平面简单图。显然平面简单图的所有有界面的度均不小于 3。

定义 8.6 称平面简单图 G 是极大平面图 (maximal planar graph), 如果在 G 中添加任一边 (它不是环, 也不是其他边的平行边) 后所得的图均非平面图。

例 8.5 中图 8.10 (c) 为极大平面图。

定理 8.3 极大平面图所有有界面都是三度的面, 无界面也是三度的。即所有面的边界均为 K_3 。

证 反设极大平面图有 4 度的面, 其边界为 $(v_1, v_2, v_3, v_4, v_1)$ 。

若该面为无界面, 那么可在无界面内联结 (v_2, v_4) ; 若该面为有界面, 那么可在有界面内联结 (v_2, v_4) 。联结后所产生的图均仍为平面图, 这与平面图的极大性冲突。

定理 8.4 顶点个数 $n \geq 4$ 的极大平面图中, 顶点的最小度数不少于 3。

证 设 v 是极大平面图 G 的任一顶点。考虑 $G - v$ 。 $G - v$ 是一平面图, v 原在 $G - v$ 的一个面内。由于 G 为一简单图, $G - v$ 的这个面的边界上至少有 3 个顶点。由 G 的极大平面性, v 与这些顶点之间都有边关联, 因此 v 至少是 3 度的顶点。

8.2.2 欧拉公式和库拉托夫斯基定理

在中学立体几何课程中, 欧拉公式是指下列关于凸多面体的顶点数 (n), 棱数 (m) 及面数 (r) 之间的关系式:

$$n - m + r = 2$$

我们设想把一个橡皮的凸四面体的底面剪破, 从此撕开四面体, 使其各面在同一平面中 (参阅图 8.12), 如果把撕破的面看作无界面, 那么图 8.12 (b) 便是一个连通平面图, 它自然也满足上述关系式。这使我们自然地想到, 对连通平面图是否依然有上述欧拉公式成立。回答是肯定的。

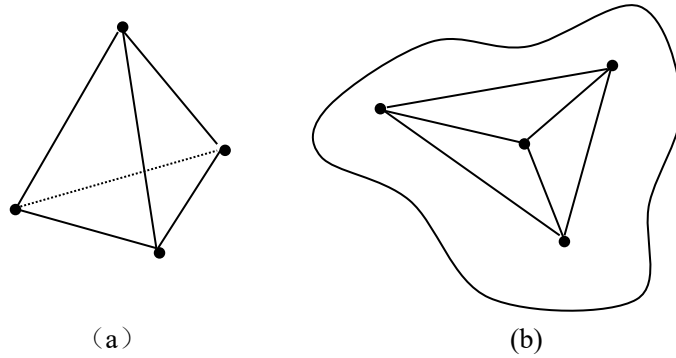


图 8.12

定理 8.5 设 G 为一平面连通图, n 为其顶点数, m 为其边数, r 为其面数, 那么

$$n - m + r = 2$$

证 对边数 m 归纳。

当 $m=0$ 时 G 为一孤立顶点, 因此 $n=1$, $m=0$, $r=1$ (一无界面), 从而 $n-m+r=2$ 。

设边数小于 m 的平面连通图均满足欧拉公式。现考虑 G (边数 $m>0$), 在 G 中任意去掉一边 e 得到图 G' 。下面对 e 的情况分别进行讨论 (参见图 8.13)

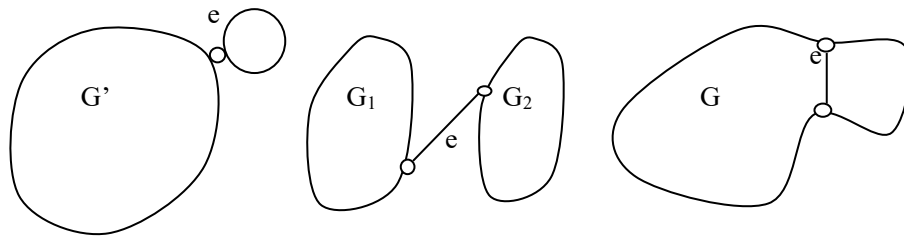


图 8.13

(1) e 为一环, 那么 G' 的顶点数、边数、面数分别是

$$n' = n, m' = m - 1, r' = r - 1$$

据归纳假设, $n' - m' + r' = 2$ 。因而

$$n - (m-1) + r - 1 = 2$$

即

$$n - m + r = 2$$

(2) $\{e\}$ 为割集, 不妨设 G' 有两个连通分支 G_1, G_2 。显然 $n_1 + n_2 = n' = n$, $m_1 + m_2 = m' = m - 1$, $r_1 + r_2 = r' + 1 = r + 1$ 。据归纳假设

$$n_1 - m_1 + r_1 = 2$$

$$n_2 - m_2 + r_2 = 2$$

从而

$$(n_1 + n_2) - (m_1 + m_2) + (r_1 + r_2) = 4$$

$$n - (m-1) + r + 1 = 4$$

即

$$n - m + r = 2$$

(3) e 关联 G 中两个顶点但 $\{e\}$ 非割集, 那么 G' 的顶点数、边数和面数分别是

$$n' = n, m' = m - 1, r' = r - 1$$

据归纳假设, $n' - m' + r' = 2$, 从而

$$n - (m-1) + r - 1 = 2$$

即

$$n - m + r = 2$$

归纳完成，定理得证。

利用欧拉公式可以获得一系列有用的推论。

定理 8.6 如果平面连通图 G 的每个面的边界都具有长度 $l(l \geq 3)$ ，那么

$$m = \frac{l(n-2)}{l-2}$$

其中 m 为 G 的边数， n 为 G 的顶点数。

证 设 G 有 k 个面。由于每一条边或者是两个面的边界，或者是一个面的两条边界（如图 8.11 中边 $\{v_6, v_7\}$ ），因此

$$l \cdot k = 2m \quad \text{或} \quad k = \frac{2m}{l}$$

将 k 代入欧拉公式，得

$$n - m + \frac{2m}{l} = 2$$

解出 m 得

$$m = \frac{l(n-2)}{l-2}$$

定理 8.7 设 G 为一平面连通简单图，其顶点数 $n \geq 3$ ，其边数为 m ，那么

$$m \leq 3n - 6$$

证 在 G 上添加边，使之成为一个极大平面图 G' 。据定理 8.3， G' 所有面的边界长度为 3，因此 G' 的边数

$$m' = \frac{3(n-2)}{3-2} = 3n - 6, \quad \text{故}$$

$$m \leq m' = 3n - 6$$

利用定理 8.7 可证明某些图是非平面图。

例 8.7 K_5 是非平面图。

证 反设 K_5 为平面图，又 K_5 是连通简单图，那据定理 8.7

$$10 \leq 3 \times 5 - 6 = 9$$

矛盾，故 K_5 非平面图。

定理 8.8 设 G 为一平面连通简单图，其顶点数 $n \geq 4$ ，边数为 m ，且 G 不以 K_3 为其子图，那么

$$m \leq 2n - 4$$

证 由于 G 是不以 K_3 为子图的简单图，因此 G 的每个面的边界长度不小于 4。

同定理 8.6 之证明， G 的 k 个面的边界长度总和应为 $2m$ ，而根据题设又知 $4k$ 不超过这一总和，因此

$$4k \leq 2m \quad \text{或} \quad k \leq \frac{m}{2}$$

据欧拉公式有

$$2 = n - m + k \leq n - m + \frac{m}{2} = n - \frac{m}{2}$$

即

$$m \leq 2n - 4$$

定理 8.7 也可仿此定理的证明，请读者自行完成。利用定理 8.8 也可证明一些图是非平面图。

例 8.8 $K_{3,3}$ 是非平面图。

证 设 $K_{3,3}$ 是平面图。显然 $K_{3,3}$ 不以 K_3 为其子图，又 $K_{3,3}$ 是连通简单图，因而定理 8.8 适用之。于是

$$9 \leq 2 \times 6 - 4 = 8$$

矛盾，故 $K_{3,3}$ 非平面图。

定理 8.9 平面连通简单图至少有一个顶点的度数不大于 5。

证 反设平面连通简单图 G 的所有顶点的度数均大于 5，因而 G 的所有顶点的度数之和至少是 $6n$ 。若 m 为 G 的边数，那么据定理 8.7 有

$$3n - 6 \geq m$$

$$6n - 12 \geq 2m \geq 6n$$

矛盾，故 G 至少有一个顶点的度不超过 5。

应当指出，欧拉公式及其上述推论，都只是平面连通图或平面连通简单图的必要条件，而不是它们的充分条件，因此只能用它们判别非平面图，不能用它们来识别平面图。那么，是否有关于平面图的判定定理呢？有，库拉托夫斯基定理给出了图为平面图的一个充分必要条件。

在介绍库拉托夫斯基定理之前，我们先引入两种对图的操作。

(1) 对边 e 的**切割**操作。设 G 中有边 $e = \{u, v\}$ ，对边 e 作切割操作是指：

(1.1) 取消边 e 。

(1.2) 增加顶点 w ，以及边 $e_1 = \{u, w\}$ ，边 $e_2 = \{w, v\}$ 。

(2) 对顶点 v 的**贯通**操作。设 G 中有二度顶点 v ，它是 $e_1 = \{u, v\}$ ， $e_2 = \{v, w\}$ 的共同端点。

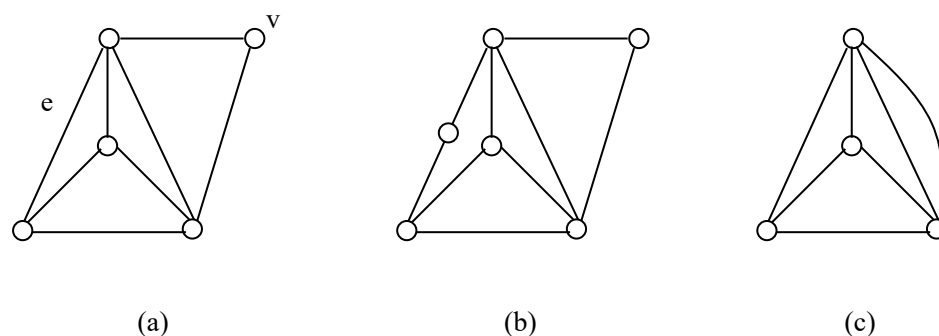
对顶点 v 作**贯通**操作是指：

(2.1) 取顶点 v 以及边 e_1, e_2 。

(2.2) 增加边 $e = \{u, w\}$ 。

切割与贯通是互逆的，两者常被称为**同胚**操作。

例 8.9 图 8.14 (a) 为图 G ，(b)，(c) 分别是边 e 切割和顶点 v 贯通操作的结果。



边 e 切割

顶点 v 贯通

图 8.14

定理 8.10 (库拉托夫斯基定理) 图 G 是平面图，当且仅当对 G 或 G 的子图作任何同胚操作后所得图均不以 K_5 及 $K_{3,3}$ 为子图。

定理的证明略去，有兴趣的读者可查阅图论教科书[5]。

例 8.10 图 8.15 (a) 为一非平面图，因为 (b) 是 (a) 的子图。而 (b) 经同胚操作后为 (c)，此乃 $K_{3,3}$ 。

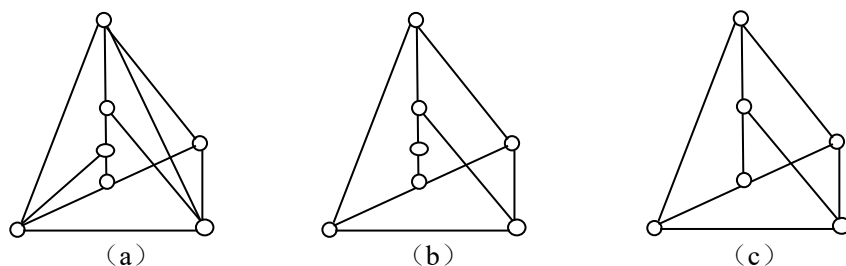


图 8.15

*8.2.3 着色问题

早在 19 世纪中期，英国数学家就提出了地图着色问题：给地图的各地域着色，要使相邻的地域具有不同的颜色，至少需要多少种颜色。显然，3 种颜色是不够的，例如图 8.16 中的 4 个地域至少要四种颜色才能作出满足上述要求的着色。另一方面，人们很快证明了，

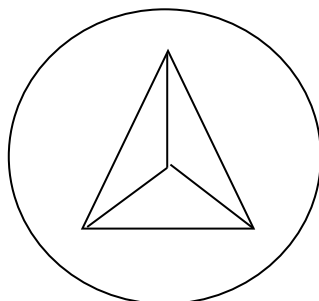


图 8.16

5 种颜色是足够的。当时英国青年盖思里 (Guthrie) 提出，用 4 种颜色即可给地图着色，使相邻区域具有不同的颜色，但他未能加以证明。于是，这给后人留下了一个著名的难题——四色问题。

地图着色问题很明显可以用平面图的面着色来刻画。为了便于讨论，我们把平面图的面着色问题转换为同等的顶点着色问题。为此，需要下列的对偶图概念。

定义 8.7 对平面连通图 G 实施下列步骤所得到的图 G^* 称为 G 的**对偶图**(dual of graph):

- (1) 在 G 的每一个面 r_i 的内部作一个 G^* 的顶点 v_i^* 。
 - (2) 若 G 中面 r_i 与 r_j 有公共边界，那么过边界的每一边 e_k 作关联 v_i^* 与 v_j^* 的一条边 e_k^* 。
- e_k^* 与 G^* 的其它边不相交。

(3) 当 e_k 为面 r_i 的边界而非 r_i 与其它面的公共边界时，作 v_i^* 的一条环与 e_k 相交 (且仅交于一处)。所作的环不与 G^* 的边相交。

例 8.10 图 8.17 (b) 是 (a) 的对偶图。(b) 中虚线部分表示原图 (a)，实线部分则是 (a) 的对偶图。

注意，当 G_1, G_2 为同构图的两种不同图示，那么它们的对偶图 G_1^* 与 G_2^* 不仅图示不同，而且可能是根本不同的图 (不同构)。这就是说，一个图的对偶图未必是唯一的。

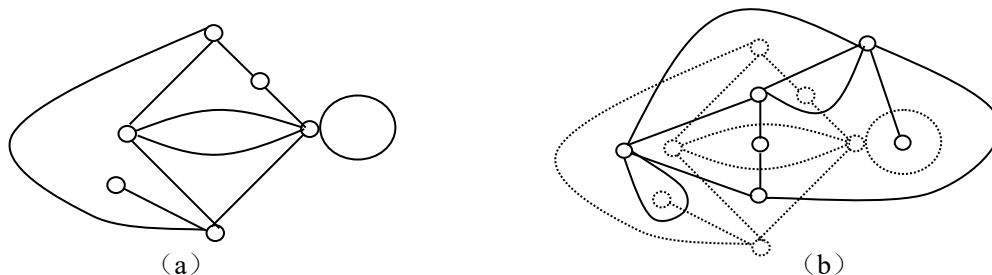


图 8.17

例 8.11 图 8.18 (a), (b) 中实线部分是两个同构的图 (图示不同), (a), (b) 中虚线部分分别表示它们的对偶图, 这两个图是不同构的, (a) 中对偶图有 5 度顶点, (b) 中对偶图却没有。

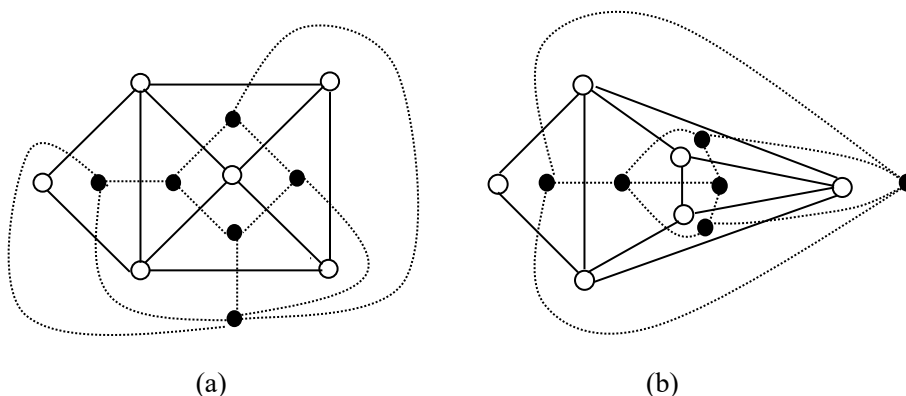


图 8.18

对偶图有许多性质, 我们关心的是:

- (1) 图 G 的面与 G^* 的顶点一一对应, 且 G 中面的度等于 G^* 中对应顶点的度。
- (2) G 中两个面有公共边界, 当且仅当 G^* 中对应顶点之间有边关联。
- (3) G 为平面图当且仅当 G^* 为平面图。

因此, 对于图的面着色问题, 可以通过研究其对偶图的顶点着色问题来解决。以下讨论图的顶点着色问题。先推广定义 7.20 中的 2-着色概念。

定义 8.8 无自环图 G 称为可 k -着色的 (k -chromatic), 如果可用 k 种颜色给 G 的所有顶点着色, 使每个顶点着一种颜色, 而同一边的两个端点着不同颜色。

定理 8.11 任何平面图都是可 5-着色的。

证 由于各连通分支可 5-着色当且仅当原图可 5-着色, 由于环和平行边与着色问题无关, 因此 可只讨论平面连通简单图。

设 G 为任一平面连通简单图, 顶点个数为 n 。对 n 归纳。

当 $n \leq 5$ 时命题显然成立。

设 $n-1$ 个顶点的平面图都是可 5-着色的。考虑 n 个顶点的图 G 。由定理 8.9, G 至少有一个顶点的度不大于 5, 设 v_0 为这样一个顶点。令 $G' = G - v_0$, 据归纳假设, G' 可 5-着色。假定 G' 已用红、黄、兰、白、黑五种颜色按要求着色, 现将 v_0 及它所邻接的边 (至多五条) 放回原处 (恢复图 G)。考虑 v_0 的着色。

(1) 设 $\deg(v_0) < 5$, 那么只要取它相邻顶点所着颜色 (至多 4 种) 之外的一种颜色给 v_0 着色, 便可完成对 G 的 5-着色。

(2) 设 $\deg(v_0) = 5$, 若与 v_0 相邻的顶点用了少于五种颜色着色, 便可完成对 G 的 5-着色。否则设 v_0 相邻顶点的着色状况如图 8.19 所示。

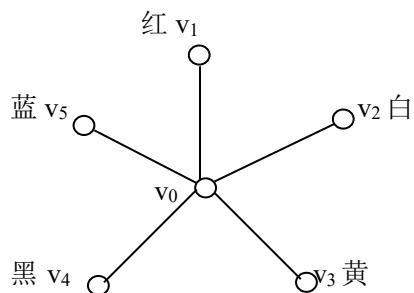


图 8.19

为叙述简明，令 RY 表示 $G - v_0$ 中所有着红、黄顶点的集合， BW 表示 $G - v_0$ 中所有着黑、白顶点的集合。考虑 RY 生成的 G 的子图 $G(RY)$ 。

若 v_1, v_3 分属于 $G(RY)$ 的两个不同的连通分支，那么只要将 v_1 所在分支的红、黄顶点的着色作一对换（从而 v_1 着黄色），便可给 v_0 着红色以完成对 G 的 5-着色。

若 v_1 和 v_3 同属于一个 $G(RY)$ 的连通分支，那么从 v_1 到 v_3 必有一条通路，其各顶点被红、黄两色相间着色。这条通路连同 v_0 便构成回路：

$$C: v_0, v_1, \dots, v_3, v_0,$$

C 把 BW 分成两部分，一部分在回路 C 之外，一部分在 C 之内。于是， BW 生成的 G 的子图也被分成了两个互不连通的部分，一部分在 C 外，一部分在 C 内，这就使 v_2, v_4 处于 BW 生成的 G 的子图的两个不同连通分支，同上将 v_2 所在分支作颜色对换，以便给 v_0 着上白色，完成对 G 的 5-着色。

归纳完成，定理得证。

8.3 树

树是一种极为简单而又非常重要的特殊图，它在计算机科学的算法设计、数据结构、网络技术、人工智能、软件工程、知识工程以及其他许多领域都有广泛的应用。

8.3.1 树的基本概念

定义 8.9 连通无回路的无向图称为**无向树**，简称为**树** (tree)。树中的悬挂点又称为**树叶** (leave)，其它结点称为**分支点** (branched node)。单一孤立结点称为**空树** (null tree)。诸连通分支均为树的图称为**森林** (forest)，树也是森林。

例 8.13 图 9.22 中 (a)，(b) 为树，而 (c) 不是树，但 (c) 为森林。

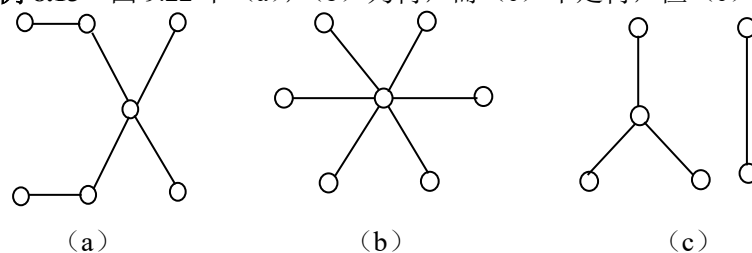


图 8.22

由于树无环也无重边（否则它有回路），因此树必定是简单图。树还有一些一目了然的性质，罗列如下，有的证明请读者完成。

定理 8.12 树和森林都是可 2-着色的，从而都是二分图。

定理 8.13 树和森林都是平面图，其面数为 1。

定理 8.14 设图 T 为一树，其顶点数、边数分别是 n, m ，那么

$$n - m = 1 \quad \text{或} \quad m = n - 1$$

证 因 T 为树，故 T 为连通平面图。据欧拉公式和 $n - m + 1 = 2$ ，即 $n - m = 1$ 。

定理 8.15 任何不少于两个顶点的树都至少有两片叶。

证 设 T 为任一树，其顶点数、边数分别为 n, m 。又设 T 中至多只有一个顶点是叶，那么

$$2m = \sum_{i=1}^n \deg(v_i) \geq 2(n-1)+1 = 2n-1$$

$$m \geq n - \frac{1}{2} > n-1$$

这与定理 8.14 矛盾。因此 T 至少有两片叶。

树还有下列 5 种等价定义形式。

定理 8.16 命题“(n, m) 图 T 为树”与下列 5 命题中的每一命题等价：

- (1) T 无回路且 $m = n - 1$ 。
- (2) T 连通且 $m = n - 1$ 。
- (3) T 无回路，但任意添加边时，T 中产生唯一的一条回路。
- (4) T 连通，但删去任一边时便不再连通 (T 的每一边均为割边)。
- (5) 任意两个不同顶点之间有且仅有一条通路。

证 记“T 为树”即“T 连通无回路”为命题 (0)。为证 (0) 与 (1), (2), (3), (4) (5) 均等价，我们证

$$(0) \Rightarrow (1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (4) \Rightarrow (5) \Rightarrow (0)$$

(0) $\Rightarrow$ (1) 这由定理 8.14 立得。

(1) $\Rightarrow$ (2) 设 T 无回路， $m = n - 1$ ，欲证 T 连通。反设 T 有 k 个连通分支 ($k \geq 2$)， $T_1, T_2, \dots, T_k$ ，它们的顶点数分别是 $n_1, n_2, \dots, n_k$ ，边数分别是 $m_1, m_2, \dots, m_k$ ，显然

$$n = \sum_{i=1}^k n_i, \quad m = \sum_{i=1}^k m_i$$

$$m_i = n_i - 1 \quad (i=1, 2, \dots, k)$$

于是

$$m = \sum_{i=1}^k m_i = \sum_{i=1}^k (n_i - 1) = n - k < n - 1$$

矛盾。因此 T 连通。

(2) $\Rightarrow$ (3) 设 T 连通且 $m = n - 1$ 。先证 T 无回路，为此对 n 归纳。

$n = 1$ 时显然 T 无回路，因这时 $m = n - 1 = 0$ 。

设顶点数为 $n-1$ 的满足题设的图无回路，考虑顶点数为 n 的图 T。去掉 T 的一悬挂点 (同定理 9.15 的证明可证，T 至少有两个悬挂点) 构成 T' 。显然 T' 仍连通，且 $m' = m - 1 = n - 2 = n' - 1$ ，因此由归纳假设 T' 无回路。在 T' 上加回所删去的悬挂点得 T，故 T 亦无回路。

再证 (3) 的第二部分，设在 T 的顶点 v_i, v_j 间添加边 e。由于 T 连通，故原本有 v_i 到 v_j 的通路，此通路连同边 e 构成 $T \cup \{e\}$ 的一条回路。若此回路不唯一，那么去掉边 e 后 T 仍有回路，与以上证明冲突。这就是说，在 T 中添加任一边后，将产生唯一的一条回路。

(3) $\Rightarrow$ (4) 留给读者自行证明。

(4) $\Rightarrow$ (5) 留给读者自行证明。

(5) $\Rightarrow$ (0) 设 T 的任何两个顶点之间有且仅有一条通路，那么 T 显然连通。T 也是无回路的，否则 T 中有回路上顶点 v_i, v_j ，使 v_i 到 v_j 有两条通路，与题设矛盾。

从以上特征性可以看出，树是“最小的连通图”，少一边便不连通；树又是“最大的无回路图”，多一边便有回路，因而树是以“最经济”的方式把其中各顶点连接起来的图。因此它作为典型的数据结构，各类网络的主干网也通常都是树的结构。

8.3.2 生成树

由上面的讨论我们知道，树是一种结构“很优秀”，也是“最经济”的特殊图，但有的连通图不是树，而由树的特性使我们想到，一个连通图 G 的“最小连通生成子图”应当是一棵树，即生成子图构成的树。这棵树对图 G 很重要，我们常称为图 G 的生成树。

定义 8.10 图 T 称为无向图 G 的**生成树** (spanning tree)，如果 T 为 G 的生成子图且 T 为树。

定理 8.17 任一连通图 G 都至少有一棵生成树。

证 若 G 无回路，则 G 本身为一树。若 G 有回路。则删去回路上任一边 e ， $G-e$ 仍连通。对 $G-e$ 重复上述讨论，直到得到 G 的无回路的连通子图——生成树。

由此可知，对于连通图 G 我们可以通过依次从回路中删边的方法得到其生成树，此方法常称为破圈法。不过，生成树通常是不唯一的。

例 8.14 图 8.23 (b)，(c) 为图 (a) 的两棵生成树。

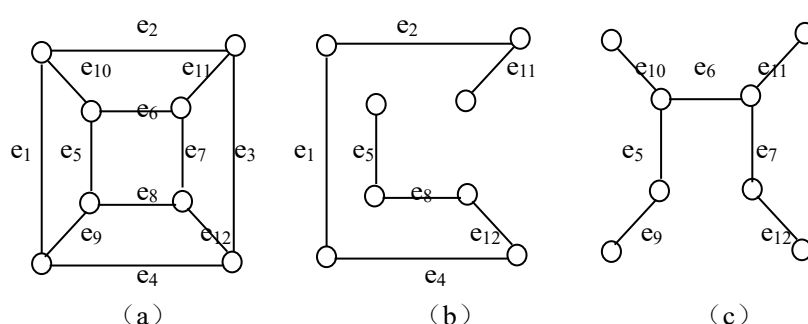


图 8.23

关于生成树，下面两个性质是容易理解的。

定理 8.18 设 G 为连通无向图，那么 G 的任一回路与任一生成树 T 的关于 G 的补 $G-T$ ，至少有一条公共边。

证 设 C 为 G 的一回路，它和 G 的生成树 T 关于 G 的补 $G-T$ 无公共边，那么 C 是 T 的子图，从而树 T 中有回路，矛盾。

定理 8.19 设 G 为连通无向图，那么 G 的任一割集与任一生成树至少有一条公共边。

证 设 S 为 G 的割集，但它和 G 的生成树 T 无公共边。那么，删去 G 中所有 S 的边后所得的图仍以 T 为子图，从而仍连通，这与 S 为割集矛盾。

联系到树的性质，我们想到，图的回路总可由一生成树上的一条路径及生成树外的一条边构成，而图的割集总可由生成树上的一条边与生成树外的若干条边组成。

定义 8.11 设 T 为图 G 的生成树，称 T 中的边为树枝 (branch)，称 $G-T$ 中的边为弦 (chord)。对每一树枝 t ， $T-t$ 分为两个连通分支 T_1, T_2 ，那么 t 及两端点分别在 T_1, T_2 中的弦组成 G 的一个割集，它被称为**枝 t -割集** (t-cut set)；而每一条弦 e 与 T 中的通路构成一回路，它被称为**弦 e -回路** (e-circuit)。

很显然， (n, m) 图 G 的任一生成树 T 恒有 $n-1$ 条边， $m-n+1$ 条弦；从而有 $n-1$ 个枝 t -割集， $m-n+1$ 个弦 e -回路 (这里的 t 指任一树枝， e 指任一弦)，它们分别称为**枝割集系**和**弦回路系**。

例 8.15 设 G 为图 8.23 (a)， G 的生成树为图 8.23 (c)，那么

- (1) $e_5, e_6, e_7, e_9, e_{10}, e_{11}, e_{12}$ ，均为树枝，共 $8-1=7$ 枝。
- (2) e_1, e_2, e_3, e_4, e_8 均为弦，共 $12-8+1=5$ 条。
- (3) 枝 e_5 -割集是 $\{e_5, e_1, e_4, e_8\}$
枝 e_6 -割集是 $\{e_6, e_2, e_4, e_8\}$

枝 e_7 -割集是 $\{e_7, e_3, e_4, e_8\}$

枝 e_9 -割集是 $\{e_9, e_1, e_4\}$

枝 e_{10} -割集是 $\{e_{10}, e_1, e_2\}$

枝 e_{11} -割集是 $\{e_{11}, e_2, e_3\}$

枝 e_{12} -割集是 $\{e_{12}, e_3, e_4\}$

(4) 弦 e_1 -回路是 (e_1, e_9, e_5, e_{10})

弦 e_2 -回路是 $(e_2, e_{10}, e_6, e_{11})$

弦 e_3 -回路是 $(e_3, e_{11}, e_7, e_{12})$

弦 e_4 -回路是 $(e_4, e_9, e_5, e_6, e_7, e_{12})$

弦 e_8 -回路是 (e_8, e_5, e_6, e_7)

现在我们来讨论枝割集系统与弦回路系统的关系。下列定理是一个准备。

定理 8.20 在连通无向图 G 中, 任一回路与任一割集均有偶数条公共边。

证 设 C 为 G 的任一回路, S 为 G 的任一割集, 从 G 中删除 S 的所有边后得到两个互不连通的子图 G_1, G_2 (如图 8.24 所示)。

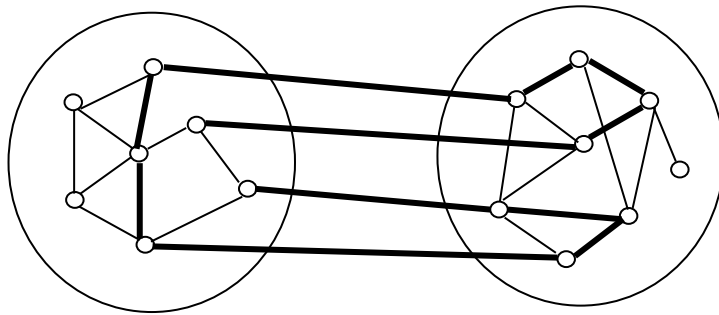


图 8.24

若回路 C 上的所有顶点都在 G_1 (或 G_2) 中, 那么, C 和 S 无公共边, 定理得证。

若回路 C 的一部分顶点在 G_1 中, 另一部分在 G_2 中, 那么 C 必定经过 S 中偶数条边 (因 C 为回路, 参阅图 8.23), 故 C 与 S 有偶数条公共边。

定理 8.21 设 G 为一连通无向图, T 是 G 的生成树,

$$S = \{e_1, e_2, e_3, \dots, e_k\}$$

为枝 e_1 -割集, 那么 e_1 必在弦 e_i -回路中 ($i = 2, 3, \dots, k$), 不在其它弦 e -回路中。 (e_1 与弦 e_i -回路恰有两条公共边 e_1 和 e_i , 而 e_1 与其它弦 e -回路无公共边。)

证 设 C 为任一弦 e_i -回路 ($i = 2, 3, \dots, k$), 例如 C 为弦 e_2 -回路, 则 e_2 在 S 与 C 中。又据定理 8.20, C 与 S 有偶数条公共边。由于 S 中只有 e_1 为树枝, 其余各边均为弦; 而 C 中只有 e_2 为弦, 其它各边均为树枝, 所以只能还有 e_1 在 S 与 C 中。即 S 与 C 恰有两条公共边 e_1 和 e_2 , 因此 e_1 在弦 e_2 -回路中。由于证明过程对一切 e_i ($i = 2, 3, \dots, k$) 均成立, 因此定理的前半部分得证。

为证定理后半部分, 设 C 为弦 e -回路, 而 $e \neq e_2, e_3, \dots, e_k$ 。而 C 中只有 e 为弦, S 中 $e_2, e_3, \dots, e_k$ 为弦, 故 $e \notin S$ 。由于 S 中只有 e_1 为树枝, 其余各边均为弦; 而 C 中只有 e 为弦, 其它各边均为树枝, $e \notin S$, 所以若 e_1 在 C 中, 那么 C 与 S 只可能有一条公共边 e_1 , 此与定理 8.20 矛盾。因此, C 与 S 无公共边, 即 $e_1 \notin C$ 。

定理 8.22 设 G 为一连通无向图, T 是 G 的生成树,

$$C = (e_1, e_2, \dots, e_k)$$

为弦 e_1 -回路, 那么 e_1 必定在枝 e_i -割集中 ($i = 2, 3, \dots, k$), 不在其它任何枝 e -割集中。

证明请读者自行完成。

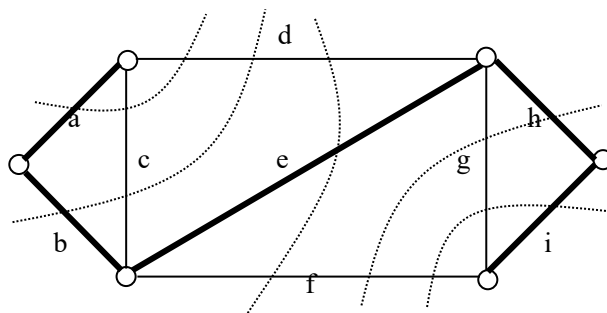


图 8.25

例 8.15 图 8.25 为一连通无向图 G ，黑粗线描出了 G 的一棵生成树，虚线表示各个枝-割集。

考虑回路 (d, a, b, e) ，它是弦 d -回路，因此， d 出现于枝 a -割集 $\{a, c, d\}$ 、枝 b -割集 $\{b, c, d\}$ 、枝 e -割集 $\{e, d, f\}$ 中，但 d 不出现于枝 h -割集 $\{h, g, f\}$ 和枝 i -割集 $\{i, g, f\}$ 中。

考虑枝 b -割集 $\{b, c, d\}$ ， b 出现于弦 c -回路 (c, b, a) 及弦 d -回路 (d, a, b, e) 中，但 b 不出现于弦 f -回路 (f, e, h, i) 和弦 g -回路 (g, h, i) 中。

由上面我们知道，一个连通图的生成树往往是不唯一的。而对于赋权图我们最关心的也是最有用的是所谓的最小生成树，下面我们来介绍它的概念及生成算法。

设 $G = \langle V, E, W \rangle$ 为连通的边赋权图， W 为 E 到非负实数集的函数。设 T 为 G 的生成树，那么 T 中各边权之和 $W(T) = \sum_{e \in T} W(e)$ 称为生成树 T 的权，权最小的生成树称为**最小生成树**。

对赋权图求最小生成树的问题，在实际应用中是经常可遇到的。以下是求最小生成树的**克鲁斯克尔 (Kruskal) 算法**。

设连通边赋权图 G 有 n 个顶点 m 条边，并设

$$W(e_1) < W(e_2) < W(e_3) < \cdots < W(e_m)$$

- (1) 设置变量 k, A 。置 k 为 1，置 A 为 $\emptyset$ 。
- (2) 若 G 的子图 $\langle V, A \cup \{e_k\} \rangle$ 不包含回路，那么置 A 为 $A \cup \{e_k\}$ 。
- (3) 当 $|A| = n-1$ 时算法终止，否则置 k 为 $k+1$ ，回到步骤 (2)。

算法的基本思想是，依边权从小到大的次序，逐边将它们放回到所关联的顶点上，但删去会生成回路的边，直至产生一个 $n-1$ 条边的无回路的子图。此法常形象称为**避圈法**。算法是正确的。为证明这一点，我们先证明下列定理。

定理 8.23 设 G 是连通边赋权图，且各边的权互不相等。若 C 是 G 中的一条回路，那么 C 上权最大的边 e 必定不在 G 的最小生成树上。

证 设 C 中权最大的边 e 在 G 的最小生成树 T 上。考虑枝 e -割集 S ， S 与回路 C 有偶数条公共边，其中之一是 e ，至少还有一条弦是公共边，记为 f 。显然， f 在 C 中， $W(f) < W(e)$ 。现将 f 放入 T 中，构成 G 的子图 H ， H 中仅有弦 f -回路。于是知 $H - e$ 为一生成树，且由于 $W(f) < W(e)$ ，可知 $W(H - e) < W(T)$ ，与 T 为最小生成树矛盾（参阅图 8.26）。

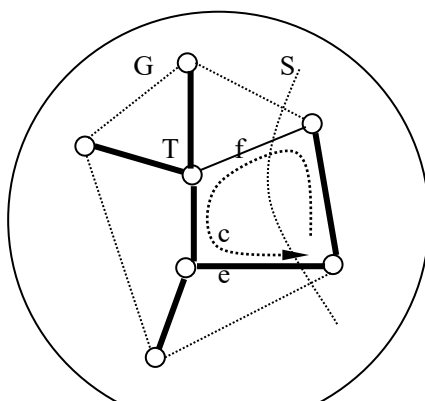


图 8.26

克鲁斯卡尔算法是正确的，因为

(1) 算法产生的图无回路，且边数 $m = n - 1$ ，据定理 8.16，此图为树。由于它含有 G 的所有 n 个顶点，因而是 G 的生成树。

(2) 设算法生成的树 T 不是最小生成树，另有最小生成树 T' ，那么至少有一边 e 在 T' 中而它不在 T 中。考虑关于生成树 T 的弦 e -回路，据算法实施过程知， e 是该回路中权最大的边。于是由定理 8.23， e 不会在 G 的最小生成树 T' 中，矛盾。因此，算法生成的树是最小生成树。

最后我们指出两点：

(1) 对于有边权相同的赋权图，克鲁斯卡尔算法依然成立。在某一回路中，若两个边权相同且最大，这时删去权最大的哪一条都一样。

(2) 利用求生成树的破圈法也可求最小生成树，但其算法与避圈法相反。即依次从图的回路中删去边权最大的边，直至没有回路结束。若在某一回路中，两个边权相同且最大，这时可删去其中的任意一条。

8.3.3 根树

递归地定义根树的概念。

定义 8.12 树 T 称为**根树** (rooted tree)，如果

(1) T 为一孤立结点 v_0 。 v_0 又被称为 T 的**树根**。

(2) $T_1, T_2, \dots, T_k$ 为以 $v_1, v_2, \dots, v_k$ 为树根的根树， T 由 $T_1, T_2, \dots, T_k$ 及与 $v_1, v_2, \dots, v_k$ 相邻的结点 v_0 所组成。 v_0 称为 T 的**树根**。

定义 8.13 在定义 8.12 中，

(1) $v_1, v_2, \dots, v_k$ 称为 v_0 的**儿子**， v_0 称为它们的**父亲**。 v_i, v_j 同为一顶点 v 的儿子时，称它们为**兄弟**。

(2) 顶点 $v_1, v_2, \dots, v_k$ 间存在一条路径，称 v_1, v_k 有顶点间的**祖孙关系**。即当 v_i 为 v_{i+1} ($i = 1, 2, \dots, k-1$) 的父亲时， v_1 是 v_k 的**祖先**， v_k 为 v_1 的**子孙**。

(3) 根树 T 自身及以它的树根的子孙为根的根树(T 的子图)，均称为 T 的**子树**(subtree)，后者又称为 T 的**真子树**。

根树除有树的一般特性外，还有下列简明的性质。

根树的每个结点都是一棵子树的树根。

除了树根，根树中每结点均为某一结点的儿子；除了树叶，根树中每一结点均为某些结点的父亲。

树根到叶有唯一的通路，这样的通路中最长一条的长度称为**树高**。

为了图示能表明根树的树根和结点间的父子关系、兄弟关系，通常约定根树的图示中：树根总画在树的顶部，同一顶点的儿子画在同一水平线上。

例 8.16 图 8.27 为一以 v_0 为根的根树，虚线框出部分为某一棵子树及该子树的子树。整个树的树高为 4。

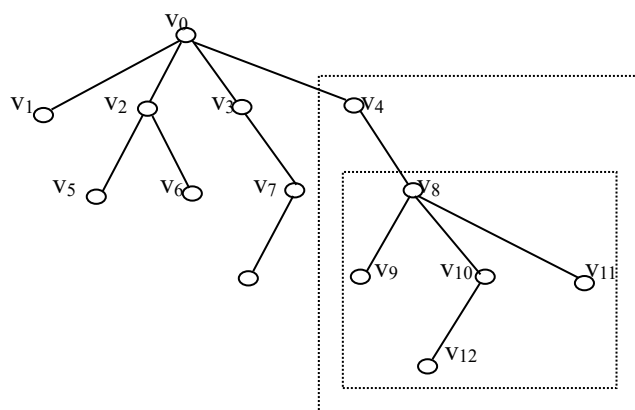
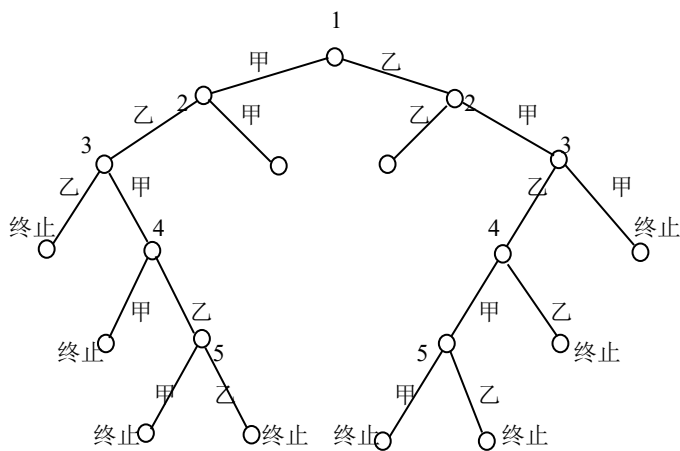


图 8.27

根树可用于表示很多种数据或逻辑关系，下面用一个例子说明这一点。

例 8.18 甲乙两人进行乒乓球赛，规定一方连胜两局或胜局首先达到 3 局者为胜方。问甲乙至少、至多要进行多少局比赛。

如果用分支结点表示一局比赛，用关联的两边表示胜负状况，标记甲的边表示甲胜，标记乙的边表示乙胜，那么可用一棵根树来描述比赛的各种可能进程，从而确定比赛至少要进行 2 局，至多要进行 5 局（见图 8.28）



8.28

图 8.28 是一种特殊的根树，完全 2 元树，这是一种最为简单而又特别有用的根树。

定义 8.14 除了树叶外，每个结点都有两个儿子的根树称为**完全 2 元树**（binary tree）。完全 2 元树有以下性质。

定理 8.24 完全 2 元树的顶点个数 n 必定是奇数。

证 完全 2 元树中除树根的度为 2 外，其它顶点的度均为 3 或 1，因此完全 2 元树度数总和为 $n-1$ 个奇数的和加 2。若 n 为偶数，则 $n-1$ 为奇数，从而树的度数的总和为奇数，但这是不可能的，因此 n 必为奇数。

定理 8.25 完全二元树中叶的数目 $l = \frac{n+1}{2}$ ，其中 $n(n \geq 3)$ 为树的顶点数。

证 设完全二元树 T 有 n 个顶点, l 片叶和 m 条边, 那么除根和叶以外它有 $n-1-l$ 个分支结点, 各为 3 度顶点, 于是

$$m = \frac{2 + 3(n-l-1) + l}{2} = n-1$$

解出 l

$$l = \frac{n+1}{2}$$

定理 8.26 完全二元树高 h 满足

$$[\log_2(n+1)-1] \leq h \leq \frac{n-1}{2}$$

这里 n 为二元树的顶点个数, $[a]$ 表示 a 的取整运算, 即小于等于 a 的最大整数。

证 如果高为 h 的完全二元树的树根到每片树叶的通路长度均为 h , 那么它的顶点数应是

$$2^0 + 2^1 + 2^2 + \cdots + 2^h = 2^{h+1} - 1$$

因此, 对一般高为 h 的完全二元树其顶点数 $n \leq 2^{h+1}-1$, 即 $2^{h+1} \geq n+1$,

因而 $h \geq \log_2(n+1)-1$, 故

$$[\log_2(n+1)-1] \leq h$$

另一方面, 如果高为 h 的完全二元树的每个分支结点的两个儿子中必有一个儿子是叶, 那么它的顶点数应是 $(h+1)+h$, 而对一般高为 h 的完全二元树其顶点数 $n \geq (h+1)+h$,

$$\text{故 } h \leq \frac{n-1}{2}.$$

现在我们来讨论非完全二元树, 简称二元树。

定义 8.15 每个结点都至多有两个儿子的根树称为**二元树** (quasibinary tree)。类似地, 每个结点都至多有 n 个儿子的根树称为 **n 元树**。对各分支结点的诸儿子规定了次序 (例如左兄右弟) 的 n 元树称为 **n 元有序树**; 若对各分支结点的已排序的诸儿子规定了在图示中的位置 (例如左、中、右), 那么该 n 元有序树又称 **n 元位置树**。2 元位置树各分支结点的左右儿子分别称为**左儿子**和**右儿子**。

例 8.19 图 8.29 (a), (b), (c) 均为 3 元树。(a), (b) 作为一般 3 元树是相同的, 但作为 3 元有序树是不同的。(b), (c) 作为一般 3 元树和 3 元有序树是相同的, 但作为 3 元位置树是不同的。

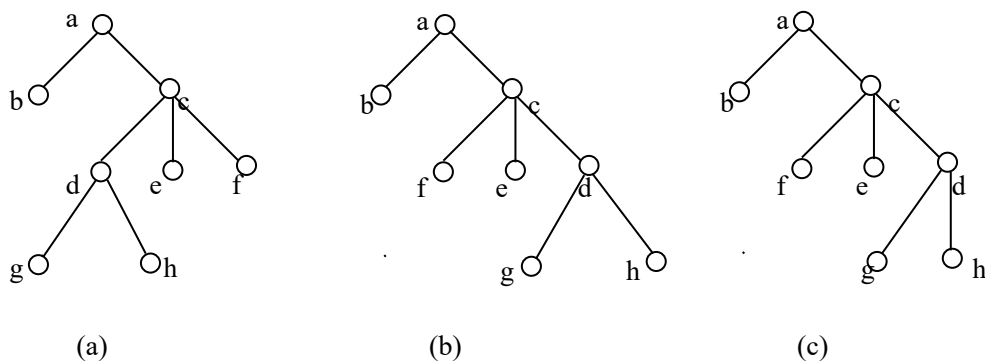


图 8.29

我们用一个例子来指出一个重要的事实：任何 n 元有序树都可以用 2 元有序树来表示，即可对任何 n 元有序树作一系列的变换，使之成为一棵 2 元位置树，从这一 2 元位置树中可得到原 n 元有序树的有关性质，甚至恢复出这一 n 元有序树。这对计算机很重要，因为计算机处理二元树是最方便的。

图 8.30 (a) 为一 4 元有序树，为了表示为 2 元位置树，只要将每个分支结点的诸儿子中的大儿子（以左兄右弟为序）保留为其儿子（左儿子），而放弃其它儿子，令二儿子为大儿子的右儿子，三儿子为二儿子的右儿子，如此等等，如图 8.30 (b) 所示。8.30 (c) 是按根树的约定画法重画图 8.30 (b) 所得的、表示图 8.30 (a) 的 2 元位置树。

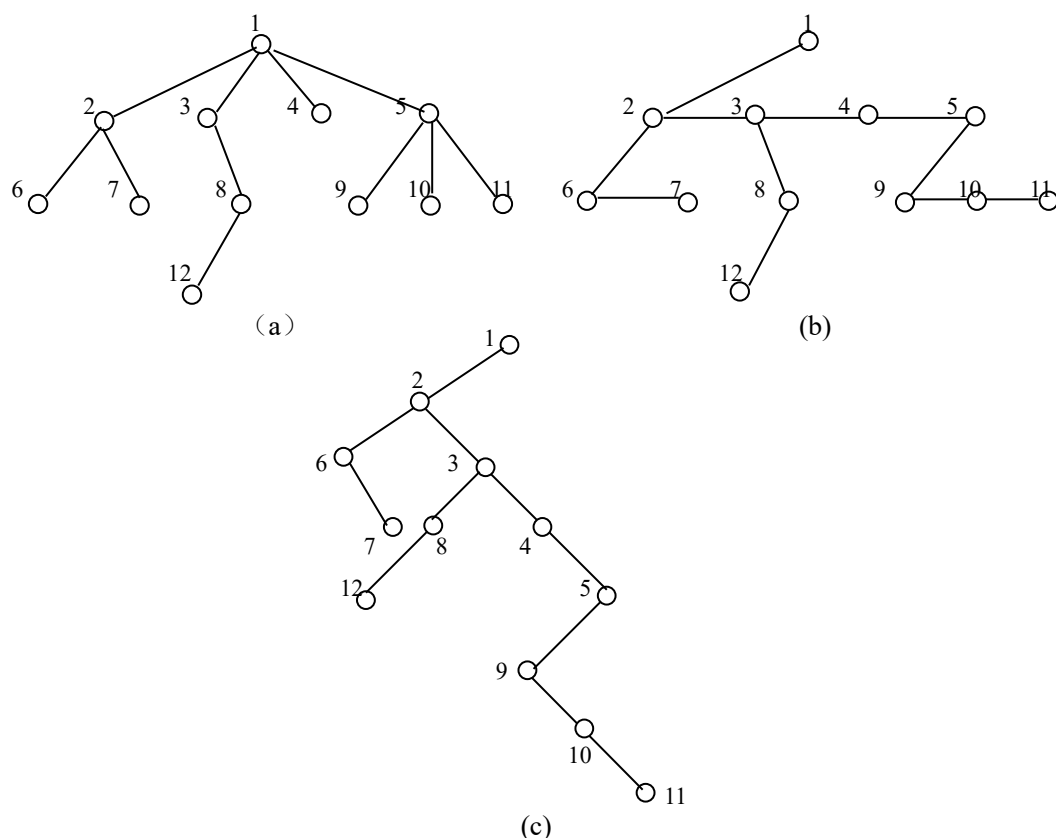
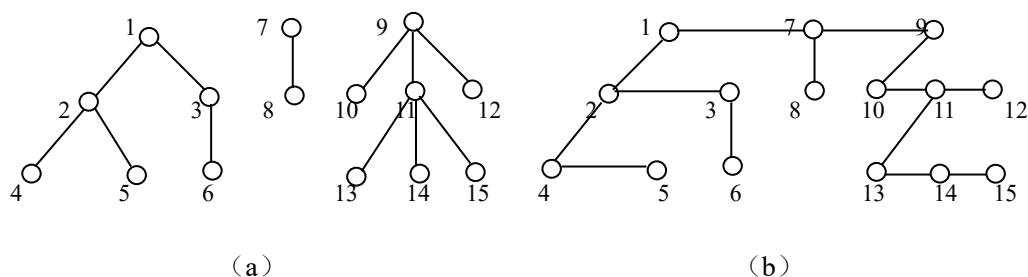


图 8.30

在作出的 2 元位置树中检索原 n 元有序树或恢复原 n 元有序树，只要把每一分支结点的左儿子看作它的大儿子，而将其右儿子看作它的弟兄，删去该分支结点到其右儿子的边，添加该分支结点的父结点到该分支结点的右儿子结点的边。

用类似的方法可用 2 元位置树来表示由 n 元有序树组成的森林（称为有序森林）。其做法是，将森林中的每一棵 n 元有序树表示为 2 元位置树，将它们的树根以父子方式连接起来，



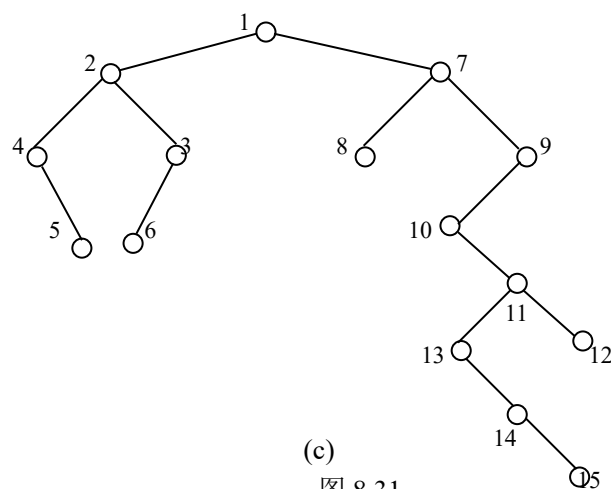


图 8.31

左父右子。图 8.31 (a) 为一 3 棵有序树组成的有序森林，(b)，(c) 为表示 (a) 的 2 元位置树。

二元位置树被大量用于数据的表示，例如表示算术表达式及各种字符串。这时，常常需要遍访每一个结点，以下是三种遍访二元树的算法。

- (1) 先根算法（前序遍历法）
 - (1.1) 访问二元树树根 r 。
 - (1.2) 如果 r 有左儿子，那么又以先根算法遍访 r 的左子树（以 r 的左儿子为根的子树）。
 - (1.3) 如果 r 有右儿子，那么又以先根算法遍访 r 的右子树（以 r 的右儿子为根的子树）。
- (2) 中根算法（中序遍历法）
 - (2.1) 如果二元树树根 r 有左儿子，那么又以中根算法遍访 r 的左子树。
 - (2.2) 访问二元树树根 r 。
 - (2.3) 如果二元树树根 r 有右儿子，那么又以中根算法遍访 r 的右子树。
- (3) 后根算法（后序遍历法）
 - (3.1) 如果二元树树根 r 有左儿子，那么又以后根算法遍访 r 的左子树。
 - (3.2) 如果二元树树根 r 有右儿子，那么又以后根算法遍访 r 的右子树。
 - (3.3) 访问二元树树根 r 。

例 8.20 算术表达式 $\frac{a * (b + c) - d \uparrow 2}{a * (b - c)}$ 可以用图 8.32 中的 2 元树来表示。其中 $*$ 表示

数乘运算， $\uparrow$ 表示指数运算， $d \uparrow 2 = d^2$ 。分别用三种算法遍访这一 2 元树，可以得到三种符号串：

由先根算法得

$$/ - * a + b c \uparrow d 2 * a - b c \quad (8-1)$$

由中根算法得

$$a * b + c - d \uparrow 2 / a * b - c \quad (8-2)$$

由后根算法得

$$abc+*d2\uparrow - abc - */ \quad (8-3)$$

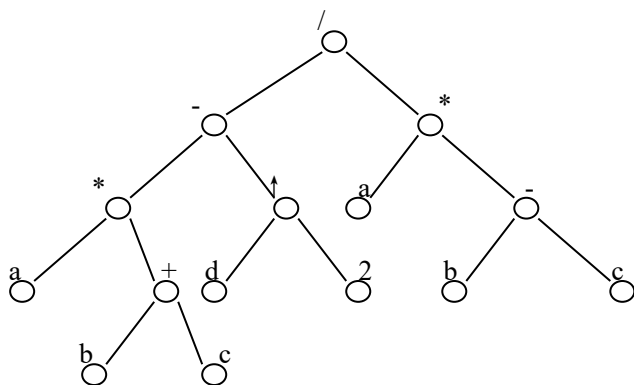


图 8.32

式 (8-1) 称为算术表达式的前置表达式 (运算符都放置在运算数据之前), 或称其为算术表达式的波兰表示。它的运算方式如式 (8-4) 所示, 与原中置表达式的运算过程一致。由于波兰表示完全不依赖括号, 所表示的运算又没有二义性, 其优越性是显而易见的。

$$\begin{array}{c} / - * a + b c \uparrow d 2 * a - b c \\ \hline \hline \hline \end{array} \quad (8-4)$$

式 (8-2) 称为算术表达式的中置表达式 (二元运算符都放置在运算数据之间)。由于遍历所得的表达式没有括号, 而二元运算符又在运算数据之间, 它所表示的运算过程的二义性就在所难免了。因此用中根算法遍历表示算术表达式的 2 元树是不适当的。

式 (8-3) 称为算术表达式的后置表达式 (运算符都放置在运算数据之后), 或称其为算术表达式的逆波兰表示。它的运算方式如式 (8-5) 所示, 与波兰表示一样, 逆波兰表示与原中置表达式的运算意义相一致。逆波兰表示也完全不依赖括号, 所表示的运算也没有二义性。

$$\begin{array}{c} abc + * d 2 \uparrow - abc - */ \\ \hline \hline \hline \end{array} \quad (8-5)$$

最后我们用两个应用根树解决实际问题的例子来结束本章的叙述。

例 8.21 有 10 个文件, 分别用 10 个不同的正整数 (2, 4, 5, 7, 8, 10, 12, 13, 15, 16) 作标记 (称为它们的键值)。为了检索方便, 可以把它们以根树的结构储存起来, 使得每一个顶点上储存的文件的键值大于它的左子树上所有文件的键值, 而小于它的右子树上所有文件的键值, 如图 8.33 所示。

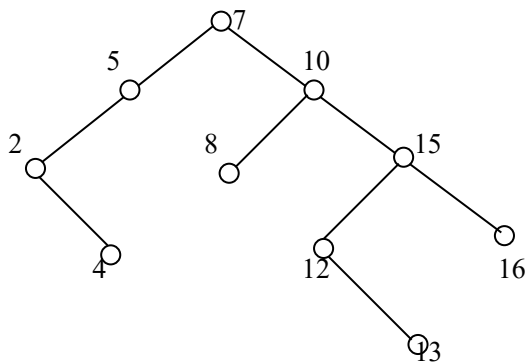


图 8.33

这时如果要检索一个文件，只要将其键值与树根的键值做比较，当它比树根的键值小时，便对左子树重复上述步骤；当它比树根的键值大时，便对右子树重复上述步骤。例如键值为 8 的文件，做两次比较便可找到。

例 8.22 有 8 枚硬币，其中恰有 1 枚是假币，假币比真币重。试用一架天平称出假币，使称量的次数尽可能地少。

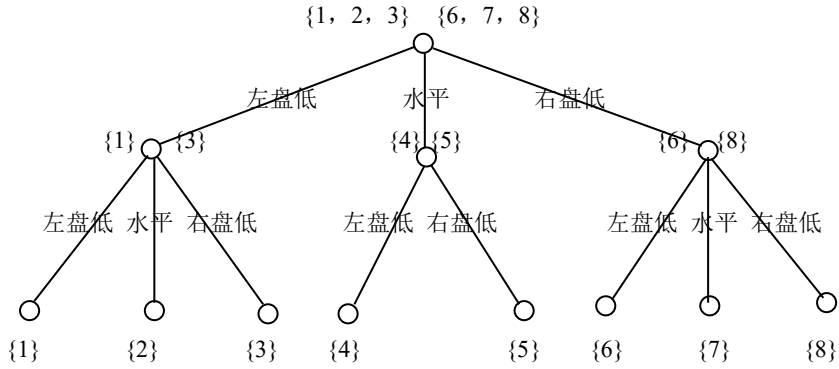


图 8.34

图 8.34 描述了称量的策略，只要称两次便可测出假币。图中八个数字表示八个硬币，结点处标记的两个集合，为一次称量中两盘所放的硬币，假币一定在一次称量中较低的盘中。